SmartWindow IoT - Sistema Autónomo de Control Climático y Seguridad

Actividad Práctica IA

Berti Rodrigo Nicolás, Fernandes Leonel Julián, Morandi Mayra Aldana, Rodriguez Fenske Axel Joel
40742053, 41799417, 41454827, 41107584
Martes, Grupo 2

Universidad Nacional de La Matanza,
Departamento de Ingeniería e Investigaciones Tecnológicas,
Florencio Varela 1903 - San Justo, Argentina

Resumen. Este informe presenta el desarrollo de SmartWindow IoT, un sistema autónomo de control climático y seguridad basado en el microcontrolador ESP32 y una aplicación móvil Android. El objetivo principal se centró en evaluar la eficiencia del paradigma de desarrollo Vibecoding total frente a la programación manual tradicional. Para ello, se utilizó la herramienta generativa Gemini (modelo 3.1 Pro) mediante prompts técnicos orientados a cumplir con restricciones arquitectónicas específicas. Se contrastaron métricas de tiempo de desarrollo, consumo de memoria interna (Heap) y uso de CPU para ambos enfoques de código. Los resultados permiten analizar la viabilidad, ventajas y limitaciones de las herramientas de IA en la ingeniería de sistemas embebidos e Internet de las Cosas (IoT).

Palabras claves: Vibecoding, Inteligencia Artificial, ESP32, IoT (Internet de las Cosas), Gemini.

1 Introducción

La En los últimos años, el Internet de las Cosas (IoT) ha transformado los entornos residenciales mediante la integración de Sistemas que automatizan tareas cotidianas de confort y protección. En este marco se diseña SmartWindow IoT, un sistema autónomo de control climático y seguridad que integra una placa de desarrollo ESP32 con sensores y actuadores físicos, vinculada de forma bidireccional mediante Wi-Fi a una aplicación móvil en entorno Android. El sistema procesa variables ambientales y comandos remotos mediante una arquitectura guiada por máquinas de estados y tareas concurrentes en FreeRTOS.

En paralelo al avance de estos sistemas, los flujos de ingeniería de software experimentan una disrupción gracias a la IA generativa y la aparición del Vibecoding.

En este paradigma el programador humano asume un rol exclusivo de supervisor y diseñador de prompts, delegando por completo en la IA la escritura del software y la subsanación de errores lógicos o de compilación.

El propósito de este informe es documentar la investigación experimental realizada por el grupo para comparar las metodologías de codificación tradicionales frente al Vibecoding Total.

Utilizando la herramienta Gemini 3.1 Pro, se generó una versión funcional del embebido. A través de esta comparativa, se busca determinar si el uso de IA técnica reduce significativamente los tiempos de entrega sin comprometer la estabilidad del hardware, evaluando el desempeño en memoria Heap y carga de CPU bajo escenarios de reposo e interacción directa.

2 Desarrollo

Herramienta de IA utilizada: **Gemini**

Modelo de IA utilizado: **3.1 Pro**

Prompt técnico utilizado:

Lenguaje de programacion: Utilizar lenguaje Wiring para el IDE Arduino

Plataforma: Desarrolar un sistema embebido ESP32 que utilice el concepto de Internet de las Cosas

Funcionalidad a implementar:
Desarrollar un sistema embebido que controle una ventana inteligente en base a la temperatura y humedad ambiente y si hay lluvia. El sistema se podra controlar mediante una app en Android comunicándose mediante un broker MQTT.

Requisitos Especificos:

Utilizar el siguiente "constantes.h":

```
#ifndef CONSTANTES_H
#define CONSTANTES_H

// Pines del microcontrolador
#define GPIO_Lluvia   34    // Sensor Lluvia
#define GPIO_DHT11    19    // Sensor humedad
#define GPIO_FC_A     25    // Final carrera Apertura
#define GPIO_FC_C     33    // Final carrera Cierre
#define GPIO_EMERG    27    // Pulsador de emergencia
```

```
#define GPIO_LED_IND  23    // Led indicador
#define GPIO_PIR      21    // Sensor pir
#define motor1PWM     5     // Pin PWM para el motor 1
#define motor1Dir2    4     // Pin de dirección 2 para el motor 1
#define motor1Dir1    2     // Pin de dirección 1 para el motor 1 (Led Azul)

// Parámetros PWM y Motor
#define PWM_FREQ          5000
#define PWM_RES           8
#define VELOCIDAD         180
#define STOP              0

// Definicion de umbral humedad y lluvia
#define HUMEDAD_UMBRAL_MAX  70
#define LLUVIA_MIN_VALIDA   500
#define HUMEDAD_MIN_VALIDA  0
#define INTERVALO           2500
#define RESET_LECTURA       0
#define PORCENTAJE_BASE     100
#define HISTERESIS_LLUVIA   100

// Tiempos y FreeRTOS
#define INTERVALO_TAREAS    20
#define STACK_SIZE          4096
#define QUEUE_SIZE          10
#define PRIORIDAD_TAREAS    1
#define PIR_TIME            5000 // Tiempo en ms
#define MAX_EVENTOS         8
#define NO_WAIT             0
#define NO_NOTIF            0
#define TIEMPO_FC           50
#define TIEMPO_REPOSO       10000
#define T_ANTIREBOTE        150
#define T_WIFI              1000
#define T_MQTT              5000
#define T_TASK              20
#define T_LOOP              100
#define T_FSM_INI           1

#define INI               0
#define FALSO             -1

// Config Wifi y MQTT
#define WIFISSID            "MGL-2.4GHz"
#define PASSWORD            "leonPlat11*"
```

```
#define MQTT_SERVER          "broker.emqx.io"
#define MQTT_NAME            "SmartWindow"
#define MQTT_USER            ""
#define MQTT_PASSWORD        ""
#define MQTT_PORT            1883

#define TOPIC_HUMEDAD            "/ventana/humedad"
#define TOPIC_TEMP               "/ventana/temp"
#define TOPIC_MODO               "/ventana/modo"
#define TOPIC_COMANDO            "/ventana/comando"  //en el modo manual, abre o cierra
#define TOPIC_ESTADO             "/ventana/estado"
#define TOPIC_EMERGENCIA         "/ventana/emergencia"
#define TOPIC_EMERGENCIA_ESTADO  "/ventana/estado/emergencia"

//Metrics
#define VALOR_INI           0
#define MIN_TOTAL_RUNTIME   1
#define PORC_TOT            100.0
#define PORC_MIN            0.0
```

Prompt Técnico: Funcionalidades del Sistema de Ventana Inteligente

A continuación se detalla el comportamiento lógico, la arquitectura y las funcionalidades del sistema embebido (ESP32 con FreeRTOS) que controla una "Ventana Inteligente". Este documento sirve como especificación técnica del comportamiento esperado.

1. Modos de Operación Base

El sistema opera mediante una Máquina de Estados Finitos (FSM) dividida en dos ramas principales:

Modo Automático: La ventana reacciona autónomamente a los estímulos ambientales (lluvia y humedad).

Modo Manual: La ventana ignora los sensores ambientales (lluvia/humedad) y solo se mueve ante comandos directos del usuario.

Nota: En ambos modos, los sistemas de seguridad (Sensor PIR y Botón de Emergencia) permanecen activos e interrumpen cualquier acción.

2. Descripción de Componentes y Funcionalidades

2.1 Sensor DHT11 (Humedad y Temperatura)

Función: Monitoreo ambiental para la ventilación automática.

Comportamiento:

Lee periódicamente la temperatura y la humedad.

En Modo Automático: Si la humedad supera un umbral máximo predefinido (HUMEDAD_UMBRAL_MAX = 70%) y no está lloviendo, el sistema emite un evento para ABRIR la ventana automáticamente y así ventilar el ambiente.

Independientemente del modo, los valores leídos se transmiten constantemente por MQTT hacia la aplicación móvil.

2.2 Sensor de Lluvia

Función: Protección del interior contra el clima.

Comportamiento:

Realiza lecturas analógicas continuas. Cuenta con una lógica de histéresis para evitar oscilaciones (falsos positivos/negativos cuando las gotas apenas tocan el sensor).

En Modo Automático: La lluvia tiene máxima prioridad ambiental. Si se detecta lluvia superando el umbral válido, el sistema emite inmediatamente el evento para CERRAR la ventana.

Si está lloviendo, se inhibe cualquier intento del DHT11 de abrir la ventana por alta humedad.

2.3 Sensor PIR (Seguridad Antipinzamiento)

Función: Evitar accidentes (atrapar manos u objetos) mientras la ventana se está cerrando.

Comportamiento:

Monitorea la presencia de movimiento en el marco de la ventana.

Si la ventana está en estado CERRANDO (ya sea manual o automáticamente) y el PIR detecta movimiento, el sistema entra inmediatamente en estado BLOQUEADO, deteniendo el motor por completo.

Al bloquearse, se inicia un temporizador interno (Timer de 5 segundos). Si hay más movimiento en ese lapso, el timer se reinicia. Si pasan 5 segundos sin detectar movimiento en el marco, el sistema asume que el obstáculo se retiró y reanuda automáticamente el cierre de la ventana.

2.4 Apertura y Cierre de la Ventana (Motor DC y Finales de Carrera)

Función: Actuación mecánica del ventanal.

Comportamiento del Motor: Controlado mediante un Puente H. Se utiliza PWM para definir una velocidad constante, y salidas lógicas (HIGH/LOW) para invertir la polaridad y cambiar entre Abrir y Cerrar.

Finales de Carrera (Límites mecánicos):

Existen dos interruptores físicos: uno para apertura total y otro para cierre total.

Cuando el motor está moviendo la ventana (abriendo o cerrando), el sistema monitorea estos interruptores. Al momento de presionarse el final de carrera correspondiente a la dirección de movimiento, el motor se DETIENE inmediatamente, alcanzando un estado de reposo seguro (ABIERTA o CERRADA).

2.5 Botón de Emergencia (Físico / Lógico)

Función: Parada de emergencia manual o remota para abortar cualquier operación.

Comportamiento:

Es un pulsador físico con lógica de anti-rebote (debouncing) por software para asegurar pulsaciones reales.

Al presionarse en cualquier momento, el motor se DETIENE en el acto, sin importar la dirección en la que iba, y el sistema queda en un estado BLOQUEADO/EMERGENCIA.

Lógica de recuperación: Si el sistema ya se encontraba detenido por una emergencia previa, presionar el botón de nuevo actúa como un "reinicio seguro", ordenando a la ventana que se CIERRE para volver a un estado inicial controlado.

3. Comunicación vía MQTT (Tópicos y Mensajería)

El sistema mantiene una conexión persistente WiFi y MQTT, escuchando comandos y emitiendo telemetría hacia una aplicación Android.

3.1 Subscripciones (Lo que el ESP32 escucha/recibe de Android)

TOPIC_MODO (/ventana/modo):

Payload esperado: "MANUAL" o "AUTOMATICO".

Acción: Fuerza el cambio de modo de la Máquina de Estados, deteniendo cualquier movimiento actual.

TOPIC_COMANDO (/ventana/comando):

Payload esperado: "ABRIR" o "CERRAR".

Acción: Inicia el movimiento del motor en la dirección indicada. (Nota: Según la FSM, este comando suele ser ignorado si el sistema está en Modo Automático).

TOPIC_EMERGENCIA (/ventana/emergencia):

Payload esperado: "EMERGENCIA".

Acción: Dispara exactamente la misma lógica de parada total que el Botón de Emergencia físico.

3.2 Publicaciones (Lo que el ESP32 envía a Android)

TOPIC_ESTADO (/ventana/estado):

Payloads: "ABRIENDO", "CERRANDO", "ABIERTA", "CERRADA", "BLOQUEADO".

Cuándo: Se emite (con bandera retain) cada vez que la Máquina de Estados cambia de condición o un final de carrera detiene el motor, para que la App siempre refleje el estado real de la estructura.

TOPIC_EMERGENCIA_ESTADO (/ventana/estado/emergencia):

Payload: "EMERGENCIA".

Cuándo: Se dispara como un flag secundario y explícito cuando el sistema entra en bloqueo específicamente debido al Botón de Emergencia (físico o remoto) o al PIR.

TOPIC_HUMEDAD (/ventana/humedad):

Payload: Porcentaje en formato numérico (ej. "65.0").

Cuándo: Constantemente, en cada ciclo de lectura válida del DHT11.

TOPIC_TEMP (/ventana/temp):

Payload: Temperatura en grados Celsius (ej. "24.5").

Cuándo: Constantemente, en conjunto con la lectura de humedad.

4. Especificación de Arquitectura y Tareas (FreeRTOS)

El diseño del sistema exige un procesamiento asíncrono basado en un Sistema Operativo de Tiempo Real (FreeRTOS).

Lectura de Sensores: Se debe utilizar una tarea dedicada (productora) de FreeRTOS encargada exclusivamente de realizar el "polling" (lectura cíclica) de los sensores. Cuando un sensor cumple su condición de activación, esta tarea encola el evento correspondiente (ej. xQueueSend).

Control de Actuadores (FSM): Se debe emplear una segunda tarea dedicada (consumidora) encargada de procesar la Máquina de Estados Finitos (FSM). Esta tarea se mantiene aislada de la lectura directa de pines; su única función es desenrutar eventos y, con base en el estado actual, comandar al motor.

5. Restricciones Críticas de Implementación (Máquina de Estados)

Para asegurar que el código cumpla con los estándares formales requeridos y evitar "código espagueti", se imponen las siguientes restricciones estrictas en la programación de la Máquina de Estados:

Función get_input() obligatoria: La extracción de eventos desde la cola de FreeRTOS (ej. xQueueReceive) no debe hacerse de forma cruda en el bucle principal de la FSM. Debe estar encapsulada estrictamente dentro de una función denominada get_input(), la cual retorna el evento válido o EVT_CONTINUE si no hay novedades.

Prohibición de sentencias if dentro de los switch: No se permite el uso de lógica condicional (if / else) dentro de los bloques case de la Máquina de Estados. Toda la lógica de transición debe estar resuelta por los propios eventos y estados.

Patrón Estricto de switch anidados: La función FSM debe estar estructurada usando exclusivamente el patrón de switch anidados:

Debe existir un primer bloque switch principal cuyo parámetro de evaluación sea la variable del estado actual.

Dentro de cada case correspondiente a un estado, debe existir obligatoriamente un segundo bloque switch (anidado) cuyo parámetro de evaluación sea el evento retornado por get_input().

Las transiciones de estado y comandos a los actuadores ocurren únicamente como sentencias ejecutadas al coincidir la combinación Estado -> Evento.

Cantidad de tokens de IA utilizados:

De entrada utilizamos 4.113 tokens, y de salida 38.000 tokens. Dando un total de **42.113 tokens.**

Tiempo para obtener una versión funcional (Manual + Vibecoding): 52hs manual + 3hs vibecoding. Dando un total de **55hs.**

Métricas CPU y memoria (Manual + Vibecoding):

- **Manual**

Caso A - Reposo:

```
=== MÉTRICAS: CASO A (REPOSO 10s SIN INTERACCIÓN) ===

Promedio del intervalo (initStats() -> finishStats())
Muestras del intervalo en milisegundos: 10003 ms

Contribución al total del sistema (promedio)
Core 0 -> Total: 100.00% | Ocupado: 5.331% | Libre (IDLE): 94.669%
Core 1 -> Total: 100.00% | Ocupado: 5.295% | Libre (IDLE): 94.705%

Estado de la memoria en el intervalo (promedio)
Heap total: 330432 bytes
Heap Libre: 208572 bytes
Heap usado: 121860 bytes
==================================================
```

Caso B – Inicio de acción:

```
=== [MÉTRICAS] CASO B: INICIO DE ACCIÓN (APERTURA) ===
[Motor] Velocidad: 180 | Dir1: 1 | Dir2: 0
[FSM] -> Cambio de Estado: 0 al 1
[Sensor] Temperatura: 14.0 °C | Humedad: 58.0 %
[Sensor] Final de carrera APERTURA presionado.
[Motor] Velocidad: 0 | Dir1: 0 | Dir2: 0

Promedio del intervalo (initStats() -> finishStats())
Muestras del intervalo en milisegundos: 1145 ms

Contribución al total del sistema (promedio)
Core 0 -> Total: 100.00% | Ocupado: 2.899% | Libre (IDLE): 97.101%
Core 1 -> Total: 100.00% | Ocupado: 2.645% | Libre (IDLE): 97.355%

Estado de la memoria en el intervalo (promedio)
Heap total: 330432 bytes
Heap Libre: 208844 bytes
Heap usado: 121588 bytes
=== [MÉTRICAS] FIN DE ACCIÓN ========================
```

- **Vibecoding**

Caso A - Reposo:

```
=== MÉTRICAS: CASO A (REPOSO 10s SIN INTERACCIÓN) ===

Promedio del intervalo (initStats() -> finishStats())
Muestras del intervalo en milisegundos: 10002 ms

Contribución al total del sistema (promedio)
Core 0 -> Total: 100.00% | Ocupado: 4.027% | Libre (IDLE): 95.973%
Core 1 -> Total: 100.00% | Ocupado: 6.343% | Libre (IDLE): 93.657%

Estado de la memoria en el intervalo (promedio)
Heap total: 329860 bytes
Heap Libre: 211704 bytes
Heap usado: 118156 bytes
================================================
```

Caso B – Inicio de acción:

```
=== [MÉTRICAS] CASO B: INICIO DE ACCIÓN (APERTURA) ===
[FSM] Transicion de Estado: 0 -> 2
[Sensor] Temp: 15.0 C, Hum: 58.0 %

Promedio del intervalo (initStats() -> finishStats())
Muestras del intervalo en milisegundos: 1331 ms

Contribución al total del sistema (promedio)
Core 0 -> Total: 100.00% | Ocupado: 3.946% | Libre (IDLE): 96.054%
Core 1 -> Total: 100.00% | Ocupado: 0.084% | Libre (IDLE): 99.916%

Estado de la memoria en el intervalo (promedio)
Heap total: 329860 bytes
Heap Libre: 211528 bytes
Heap usado: 118332 bytes
=== [MÉTRICAS] FIN DE ACCIÓN ========================
```

Código en Git (Manual + Vibecoding): https://github.com/UNLAM-SOA/2026-SOA-Q1-M2.git

3 Conclusiones

Rúbrica de funcionalidad **EMBEBIDO ESP32**

RUBRICA FUNCIONALIDAD EMBEBIDO ESP32

CRITERIOS	SUFICIENTEMENTE LOGRADO (A)	MEDIANAMENTE LOGRADO (B)	INSUFICIENTEMENTE LOGRADO (C)	Puntaje	Eval
FUNCIONA SIN ERRORES	Compila y funciona sin errores.	Funciona con errores menores o warnings.	Presenta errores que impiden su uso.	A: Hasta 20 pts. B: Hasta 15 pts. C: 0	A: 20
EMPLEA TAREAS DE FREERTOS	-Utiliza varias tareas de FreeRTOS	-Utiliza una sola tarea con FreeRTOS	-No utiliza FreeRTOS	A: Hasta 20 pts. B: Hasta 10 pts C: 0	A: 20
ESPERAS BLOQUEANTES	-Utiliza temporizadores de FreeRTOS	-Utiliza temporizadores sin FreeRTOS	-Utiliza esperas que bloquean al embebido	A: Hasta 20 pts. B: Hasta 15 pts. C: 0	A: 16
UTILIZA LOS SENSORES Y ATUADORES SOLICITADOS	utiliza la cantidad de sensores y actuadores solicitados de manera eficiente	-utiliza la cantidad de sensores y actuadores de manera poco efectiva	-no utiliza la cantidad de sensores y actuadores solicitados o no funcionan de manera correcta.	A: Hasta 20 pts. B: Hasta 12 pts. C: 0	A: 20
SE COMUNICA CON ANDROID	Comunicación estable en ambos sentidos.	Comunicación parcial o inestable.	La comunicación falla.	A: Hasta 20 pts. B: Hasta 15 pts. C: 0	A:20
TOTAL				**100**	**96**

4 Referencias

1. *SOA-UNLaM*. Disponible en: http://www.soa-unlam.com.ar/wiki/index.php/PUBLICO:Portada

2. "*Clase de Vibecoding y Métricas con IA*" diapositivas de la asignatura *Sistemas Operativos Avanzados, Departamento de Ingeniería e Investigaciones Tecnológicas, Universidad Nacional de La Matanza, San Justo, Argentina, 2026*.